Motivation

- When you want to store several properties about an entity, you need to define several variables.
  - Eg: If you want to store the name, ID, department, class, and GPA of a student, you need to define 5 variables as follows:

```
char name[41];
long int id;
char department[16];
short int class;
float GPA;
```

- Together with all other variables you need to define, it is difficult to keep track of this info; there is nothing that shows the association between these variables.
- Structures help you to better organize your code. (It will be more obvious when we start discussing arrays of structures.)
- Structures also open up the way for object-oriented programming.

Structures

- Syntax:

```
struct structure_name {
   field_definition(s)
   ...
} variable_list;
```

where

- **structure_name** is optional, but without it you will not be able to refer to this type once again (for new variables and parameter definitions, etc.)
- There should be at least one field, but of course it should typically be two or more.
- **variable_list** is optional. You may define the variables later on.

Example #1

```
struct stu_info {
    char name[41];
    long int id;
    char dept[16];
    short int class;
    float gpa;
} stu1, stu2;
```

- Now all information about students #1 and #2 are gathered under two variables, **`stu1`** and **`stu2`**.

Structures

- Structure is a user-defined type (like enumerated types).
- Note that **`stu_info`** is the name of the structure type, but **`stu1`** is the name of a variable of that type.
- Analogy:
  - **`stu_info`** is the name of a type, just like **`int`**.
  - **`stu1`** is the name of a variable of the given type.
- Therefore, you cannot assign any value to **`stu_info`**; it is the type, not the variable.

Accessing Fields of a struct

- You may access a field of a structure as follows:

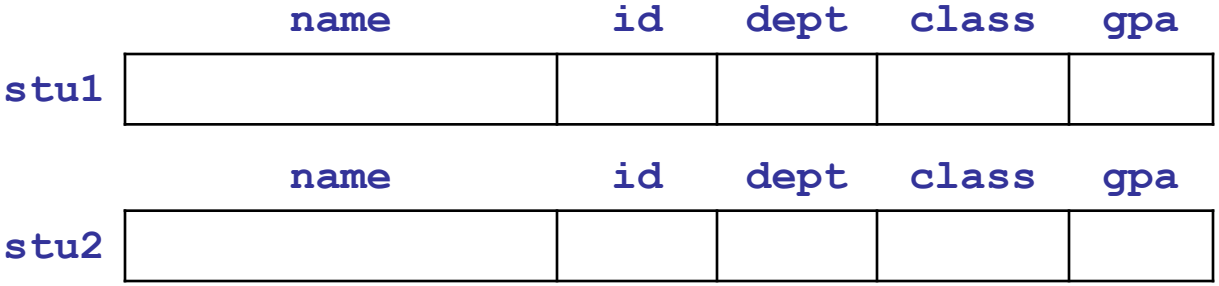
```
structure_variable_name.field_name
```

- You cannot use a field name alone. A field name makes sense only in the context of a structure variable
  - i.e., `id=123;` is wrong (`id` is not defined)

    `stu1.id=123;` is correct

Accessing Fields of a struct

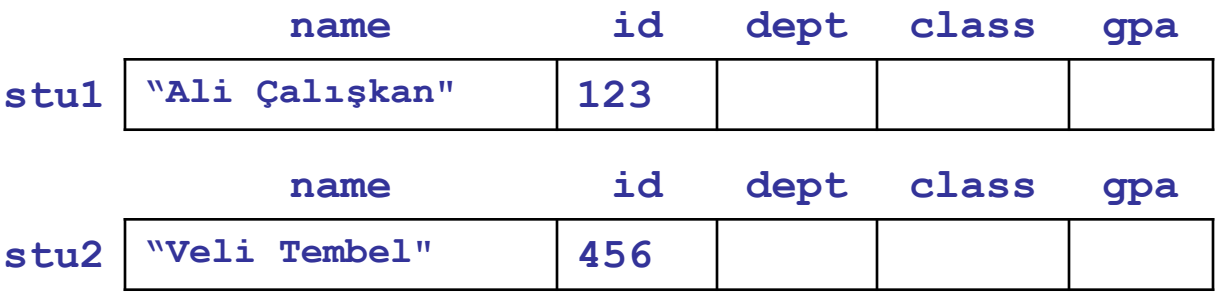
- Note that `stu1` and `stu2` are two separate variables, each with 5 fields.



- You can refer to individual fields of `stu1` and `stu2` as follows:

```
strcpy(stu1.name, "Ali Çalışkan");    stu1.id=123;
strcpy(stu2.name, "Veli Tembel");    stu2.id=456;
```

- Then, the variables will be as follows:



Defining struct Variables

- There are two ways of defining a struct variable:
  - Define the variable while defining the structure.

```
struct {
   char name[21];
   int credit;
} CSE1145, CSE1146;  /*Define both type and variables*/
```

    - Note that the optional struct name has been omitted.
  - Define the variable after the structure has been defined.

```
struct course_type {
   char name[21];
   int credit;
};  /*Define only the type*/
struct course_type CSE1145, CSE1146; /*Define vars*/
```

    - Note that the optional struct name cannot be omitted here since we need the struct name later to define variables of that type.
    - Also, note that you cannot simply say `course_type`; you have to say `struct course_type`.

struct as a Field of Another struct

- You may have a struct that has a field of struct type.
- Eg:

```
struct A_type {
   int m, n;
};

struct B_type {
   int f;
   struct A_type g;
} t;
```

  - `t` has two fields: `f` and `g`. `f` is of type `int`, `g` is of type `struct A_type`. Thus, `t` has the following fields (and subfields):

```
t.f
t.g
t.g.m
t.g.n
```

Example #2

- Define a type for a point in two-dimensional space.

```
struct point_type {
    int x, y;
};
```

Example #3

- Define types for a triangle and a rectangle.

```
struct triangle_type {
   struct point_type A, B, C;
};

struct triangle_type t={{1,3},{2,4},{1,6}};

struct rectangle_type {
   struct point_type A, B, C, D;
};
```

Example #4

- Define a type for a cube.

```
struct point3_type {
   int x,y,z;
};

struct cube_type {
   struct point_type c1,c2,c3,c4,c5,c6,c7,c8;
};
```

Can you find a better representation for a cube?

Initializing struct Variables

- You may initialize struct variables during definition (in a way similar to arrays).

```
struct A_type {
   int m, n;
} k={10,2};

struct B_type {
   int f;
   struct A_type g;
} t={5,{6,4}};
```

- The following kind of initialization is wrong.

```
struct A_type {
   int m=10, n=2;
} k;
```

Initializing struct Variables

- You cannot use the `{...}` format for initialization after initialization (just as in arrays),
- ie,

```
struct A_type {
        int m, n;
    } k;
    ...
    k={10,2};
```

is wrong.

struct as a Parameter

- You may have a struct parameter (just like any other parameter).

```
void func1(struct A_type r)
{   struct A_type s;
    s=r;
    s.m++;
}
```



- You may also have a variable parameter (CSE1146 Topic).

```
void func2(struct A_type *p)
{   (*p).m=10;
    (*p).n=3;
}
```

Example #5

```
struct complex {
   float real;
   float imaginary;
} c={5.2,6.7}, d={3,4};

struct complex add(struct complex n1, struct complex n2)
{  struct complex r;

   r.real = n1.real + n2.real;
   r.imaginary = n1. imaginary + n2. imaginary;
   return r;
}
```

Field of a struct pointer (CSE1146 Topic)

- In the previous slide in `func2()`, instead of `(*p).m`, we could have used `p->m`.
  - The "`->`" operator works only if "`p`" is a pointer to a struct, which has a field named `m`.
- In other words, you cannot say `r->m` in `func1()`.

Field of a struct pointer (CSE1146 Topic)

```
void func2(struct A_type *h)
{
    (*h).m=5; /* Equivalent of "h->m=5;"*/
}

int main()
{   struct A_type k={1,2};
    func2(&k);
    printf("%d  %d\n", k.m, k.n);
}
```

struct as the Return Type

- You may also have a return type of struct.

```
struct A_type func4()
{   struct A_type s={10,4};
    ...
    return s;
}
```

Exercise #1

- Write a function that takes a parameter of `struct stu_info` and initializes it.

Array of struct

- Array of struct is straight forward. It is like array of any other type.

```
int          number[100];
struct stu_info   class[100];


number[3] = 42;
class[3].id  = 42;
```

Example #6

- Re-define the type for a cube.

```
struct cube_type {
    struct point_type corner[8];
};
```

Example #7

- Define the cube by its edges rather than its corners.

```
struct line_type {
    struct point3_type start, end;
};

struct cube_type {
    struct line_type edge[8];
};
```

Example #8

- Write a program that collects info about 100 students in class and finds the average of their GPAs.

Example #8 *(cont'd)*

```
#include <stdio.h>

struct stu_info {
   char name[41];
   long int id;
   char dept[16];
   short int class;
   float gpa;
};

void exercise_1(struct stu_info s[])
{...}

int main()
{  struct stu_info student[100];
   int i;   float avg=0;

   exercise_1(student);
   for (i=0; i<100; i++)
      avg += student[i].gpa;
   printf("%f\n",avg/=100);
   return 0;
}
```

Size of a struct

- Assume in your system and short int occupies 2 bytes and an int occupies 4 bytes.
- What is the size of the following struct?

```
struct A {
    short int m;
    int n;
    char k;
};
```

- It is at least **2+4+1=7** bytes, but could be even larger → Depends on your system.

Struct Example

- **Closest pair Brute-Force Algorithm**

  -Find the two closest points in a set of *n* points (in the two-dimensional Cartesian plane)

- **Brute-force algorithm**

  - Compute the distance between every pair of distinct points
  - return the indexes of the points for which the distance is the smallest

```
ALGORITHM BruteForceClosesPoints(P)
 //Input: A list P of n (n>=2) points P1 =(x1, y1), ..., Pn=(xn, yn)
 //Output: Indices index1 and index2 of the closest pair of points
 d_min<-infinite
 for i=1 to n-1 do
      for j=i+1 to n do
               d<-sqrt((x_i-x_j)^2 + (y_i-y_j)^2)
               if (d<d_min)
                        d_min<-d;
                        index1<-i;
                        index2<-j;
 return index1, index2;
```

union

- When you use a struct, all fields store values simultaneously.
- Sometimes, it is necessary to store one field or the other exclusively (i.e., not both).
  - That is why you need a union.

union

- The syntax is very similar to struct:

```
union union_name {
    field_definition(s)
    ...
} variable_list;
```

- Eg:

```
struct M {
    int a;
    float b;
    double c;
};
```

- The difference is that a single value is stored.
- The size of the union is the size of the largest field (rather than the sum of the sizes of all fields).

Example #9

- Consider the following struct:

```
struct staff_info {
    char name[41];
    long int SSid;
    enum {assist, prof, personnel} status;
    int salary;
};
```

- Write a program that fills in an array of 100 elements where each element could be of type stu_info or staff_info.

Example #9 *(cont'd)*

```
#include <stdio.h>

struct person_info {
   enum {student, staff} type;
   union {
      struct stu_info {
         char name[41];
         long int id;
         char dept[16];
         short int class;
         float gpa;
      } student;
      struct staff_info {
         char name[41];
         long int SSid;
         enum {assist, prof, personnel} status;
         int salary;
      } staff;
   } info;
};
```

Example #9 *(cont'd)*

```
void read_student(struct stu_info *s)
{...}
void read_staff(struct staff_info *s)
{...}

int main()
{  struct person_info person[100];
   int i;

   for (i=0; i<100; i++)
   {  printf("Enter 0 for student, 1 for staff: ");
      scanf("%d", &person[i].type);
      if (person[i].type==student)
         exercise_1_student(&person[i].info.student);
      else
         exercise_1_staff(&person[i].info.staff);
   }
   return 0;
}
```

typedef

- You may define new names for existing types using typedef.
  - Note that typedef does not create a new type.
- Syntax:

```
typedef existing_type_name new_type_name(s);
```

- Eg:

```
typedef int tamsayi, int_arr[10];
```

- Now, you can do the following:

```
tamsayi i, j, arr[50];
int_arr a;
i=10;   j=35;   arr[3]=17;
a[2]=15;
```

typedef

- typedef is mostly useful for structures to avoid using the word "struct" in front of the structure name.
- Eg:

```
typedef struct A_type  A_t;
typedef struct {
   int x, y;
} nokta_t, noktalar_t[10];
nokta_t n;
noktalar_t N;
n.x = 5;
N[4].x = 8;
```